

Git & GitHub Interview Questions

100 Interview Questions + Practical Commands + Senior-Level Scenarios

Interview focus: Git fundamentals, branching, merge/rebase, reset/revert, GitHub collaboration, internals, recovery, and real-world senior scenarios.

Command	Primary purpose
git add	Stage changes
git commit	Create a commit from staged changes
git diff	Inspect unstaged changes
git fetch	Download remote updates without integrating
git pull	Fetch and integrate remote updates
git merge	Combine branch histories
git rebase	Replay commits onto a new base
git reset	Move references / modify index and working tree
git revert	Create a commit that reverses another commit
git restore	Restore files / unstage changes
git stash	Temporarily store uncommitted changes
git cherry-pick	Apply a specific commit to current branch
git reflog	Recover local reference movements

1. Git Fundamentals

1. What is Git?

Git is a distributed version control system used to track changes, manage branches, collaborate with developers, and maintain project history.

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git init
git status
git add .
git commit -m "Initial commit"
```

2. Git vs GitHub

Git is the version control tool, while GitHub is a platform that hosts Git repositories and provides collaboration features such as pull requests, code reviews, issues, permissions, and GitHub Actions.

```
git init
git clone <repository-url>
git remote -v
git push origin main
```

3. What is a Git repository?

A Git repository is a directory tracked by Git. It contains project files and Git metadata.

```
mkdir my-project
cd my-project
git init
git status
```

4. What is the .git folder?

.git is Git's internal database and metadata directory. It contains objects, references, HEAD, configuration, the staging index, and logs.

```
ls -la
ls .git
git rev-parse --git-dir
git rev-parse --show-toplevel
```

5. What does git status do?

It shows the current state of the working directory and staging area, including modified, staged, untracked files and the current branch.

```
git status
git status --short
```

2. Git Workflow

6. Explain the Git three-stage workflow.

Working Directory → git add → Staging Area → git commit → Git Repository.

```
git status
git add app.js
git status
git commit -m "Add login functionality"
```

7. What is the staging area?

The staging area lets you choose exactly which changes will go into the next commit.

```
git add file1.js
git add file1.js file2.js
git add .
git add -u
```

8. What is an atomic commit?

An atomic commit represents one logically complete change. Atomic commits make code review, debugging, reverting, and history management easier.

```
git add login.js
git commit -m "Add login validation"
```

9. What makes a good commit message?

A good commit message is clear, short, specific, and descriptive.

```
git commit -m "Add JWT authentication"
git commit -m "Fix login validation"
```

10. What is git log?

git log displays commit history.

```
git log
git log --oneline
git log --graph
git log --decorate
git log --all
git log --oneline --graph --decorate --all
git log -- app.js
git log --author="Priyanka"
```

3. .gitignore

11. What is .gitignore?

.gitignore tells Git which files and directories should not be tracked.

```
node_modules/
.env
.env.*
.next/
dist/
coverage/
git check-ignore -v .env
```

12. Why should .env files usually be ignored?

Environment files may contain sensitive values such as DATABASE_URL, JWT_SECRET, API_KEY, and AUTH_SECRET.

```
git check-ignore -v .env
```

13. I accidentally committed .env. What should I do?

Remove the file from tracking, add it to .gitignore, commit the change, and immediately rotate/revoke any exposed credentials if the file was pushed.

```
git rm --cached .env
git add .gitignore
git commit -m "Stop tracking environment file"
```

4. Branches

14. What is a Git branch?

A branch is a movable reference to a commit that allows independent development.

```
git branch
git branch feature/login
git switch feature/login
git switch -c feature/login
```

15. What is HEAD?

HEAD represents your current checked-out position.

```
git status
git log -1
git symbolic-ref --short HEAD
cat .git/HEAD
```

16. What is detached HEAD?

A detached HEAD occurs when HEAD points directly to a commit rather than a branch. Create a branch to preserve work.

```
git checkout abc123
git status
git switch -c recovery-branch
```

17. git checkout vs git switch

git checkout is an older multi-purpose command. git switch is designed specifically for branch operations; git restore is used for restoring files.

```
git checkout feature/login
git checkout -b feature/login
git switch feature/login
git switch -c feature/login
git restore file.js
```

18. How do you create and switch to a branch?

Modern syntax uses git switch -c; the older equivalent is git checkout -b.

```
git switch -c feature/login
git checkout -b feature/login
```

19. How do you delete a branch?

Local branches can be deleted with -d or force-deleted with -D. Remote branches can be deleted with git push origin --delete.

```
git branch -d feature/login
git branch -D feature/login
git push origin --delete feature/login
git branch -a
git branch -r
```

5. Merge

20. What is Git merge?

Merge combines changes from one branch into another.

```
git switch main
git merge feature/login
```

21. What is a fast-forward merge?

A fast-forward merge occurs when the target branch has not diverged. Git can simply move the target branch pointer forward without creating a merge commit.

```
git switch main
git merge feature
```

22. What is a merge commit?

When branches have diverged, Git may create a new commit with two parents.

```
git switch main
git merge feature
```

23. What is a merge conflict?

A merge conflict occurs when Git cannot automatically determine how competing changes should be combined.

```
git status
```

24. How do you resolve a merge conflict?

Resolve conflict markers manually, stage the resolved files, and commit. You can abort the merge if needed.

```
git merge feature/login
git status
git add .
git commit
git merge --abort
```

6. Git Diff

25. What does git diff do?

git diff shows differences between Git states.

```
git diff
git diff --staged
git diff main feature/login
git diff abc123 def456
```

26. git diff vs git diff --staged

git diff shows working-directory changes not staged. git diff --staged shows changes already staged compared with the latest commit.

```
git diff
git diff --staged
```

27. How do you compare two commits?

Use git diff with the two commit IDs.

```
git diff <commit1> <commit2>
git diff a1b2c3 d4e5f6
```

7. Git Stash

28. What is git stash?

git stash temporarily stores uncommitted changes so you can switch branches or perform other work without committing incomplete changes.

```
git stash
git switch main
git stash pop
git stash -u
```

29. git stash pop vs git stash apply

pop applies the stash and removes it if successful. apply restores it but keeps it in the stash list.

```
git stash pop
git stash apply
```

30. How do you see all stashes?

Use git stash list. A specific stash can be applied, dropped, or all stashes can be cleared.

```
git stash list
git stash apply stash@{1}
git stash drop stash@{1}
git stash clear
```

8. Undoing Changes

31. What is git restore?

git restore is primarily used to restore files or unstage files.

```
git restore app.js
git restore --staged app.js
git restore --source=HEAD --staged --worktree app.js
```

32. What is git reset?

git reset moves HEAD/branch references and can modify the staging area and working tree depending on the mode.

```
git reset --soft HEAD~1
git reset --mixed HEAD~1
git reset --hard HEAD~1
```

33. Explain reset --soft, --mixed, and --hard.

--soft resets the commit while keeping staging and working files. --mixed resets the commit and unstages changes while keeping working files. --hard resets the commit, staging area, and working files.

```
git reset --soft HEAD~1
git reset --mixed HEAD~1
git reset --hard HEAD~1
git reset HEAD~1
```

34. Why is git reset --hard dangerous?

It can discard working-directory changes and move the branch backward.

```
git reset --hard HEAD~1
```

35. What is git revert?

git revert creates a new commit that reverses the effect of an earlier commit. It is generally safer for shared branches.

```
git revert abc123
```

36. reset vs revert

Reset moves references and can rewrite history. Revert creates a new reversing commit and preserves existing history.

```
git reset HEAD~1
git revert HEAD
```

9. Git Amend

37. What is git commit --amend?

It modifies the most recent commit instead of creating another commit.

```
git commit --amend -m "Correct commit message"
git add forgotten-file.js
git commit --amend --no-edit
```

38. When should you avoid amend?

Avoid amending commits already shared with other developers because amend changes the commit identity and rewrites history.

10. GitHub & Remote Repositories

39. What is a Git remote?

A remote is a reference to another repository, commonly hosted on GitHub.

```
git remote -v
git remote add origin <repository-url>
git remote remove origin
git remote rename origin upstream
```

40. What is origin?

origin is the conventional default name for the remote repository. It is simply an alias and can be renamed.

```
git remote -v
git remote rename origin github
```

41. What is git clone?

git clone creates a local copy of a remote repository, including its history and remote configuration.

```
git clone <repository-url>
git clone <repository-url> my-project
```

42. What does git push do?

git push uploads local commits to a remote repository.

```
git push origin main
```

43. What does git push -u origin main mean?

-u establishes the upstream tracking relationship between the local and remote branch.

```
git push -u origin main
git push
git pull
```

11. Fetch vs Pull

44. What is git fetch?

git fetch downloads remote changes without integrating them into the current branch.

```
git fetch origin
git fetch --all
```

45. What is git pull?

git pull generally fetches remote changes and then integrates them into the current branch. Depending on configuration, integration may be merge or rebase.

```
git pull
git pull --rebase
```

46. fetch vs pull

fetch lets you inspect incoming changes before integration; pull fetches and integrates immediately.

```
git fetch origin
git log HEAD..origin/main
git diff HEAD origin/main
git merge origin/main
git rebase origin/main
```

12. Remote Tracking Branches

47. What is origin/main?

origin/main is a remote-tracking reference representing your last fetched knowledge of the remote main branch.

```
git branch -r
```

48. What happens when you run git fetch?

Git updates remote-tracking references such as origin/main and origin/feature/login without automatically modifying your local working branch.

```
git fetch origin
```

13. GitHub Collaboration

49. What is a Pull Request?

A Pull Request is a request to merge changes from one branch into another, usually with code review and CI checks.

```
git push -u origin feature/login
```

50. What is a feature branch workflow?

Create a feature branch, develop, commit, push it, then create a Pull Request.

```
git switch main
```

```
git pull
git switch -c feature/login
git add .
git commit -m "Add login validation"
git push -u origin feature/login
```

51. Why are feature branches useful?

They isolate development, support parallel work, make reviews easier, reduce risk to main, and support independent CI/testing.

52. What are branch protection rules?

Branch protection can require Pull Requests, approvals, passing CI/status checks, and can prevent direct or force pushes to important branches.

53. What is a fork?

A fork is a server-side copy of another GitHub repository under your account. It is common in open-source workflows when you do not have direct push permission.

```
git clone <your-fork-url>
git remote -v
git switch -c feature/my-change
git push -u origin feature/my-change
git remote add upstream <original-repository-url>
git fetch upstream
```

14. Merge vs Rebase

54. What is Git rebase?

Rebase moves or replays commits onto another base. The replayed commits receive new commit IDs.

```
git switch feature
git rebase main
```

55. Merge vs rebase

Merge preserves existing history and may create a merge commit. Rebase rewrites commits onto a new base and generally creates a linear history.

```
git merge main
git rebase main
```

56. Why is rebasing dangerous?

Rebase rewrites commit history. If other developers already based work on those commits, rewriting them can cause confusion and duplicate histories.

57. What is the golden rule of rebasing?

Do not rebase commits that other people have already based their work on. Rebase private/local branches; avoid rewriting shared history.

58. How do you handle conflicts during rebase?

Resolve the conflicted files, stage them, continue the rebase, or abort/skip as appropriate.

```
git rebase main
git status
git add .
git rebase --continue
git rebase --abort
git rebase --skip
```


15. Interactive Rebase

59. What is interactive rebase?

Interactive rebase allows you to modify a sequence of commits, including rewording, squashing, fixing up, dropping, editing, and reordering.

```
git rebase -i HEAD~5
```

60. What is squashing?

Squashing combines multiple commits into one, producing cleaner feature history before merging.

```
git rebase -i HEAD~4
```

61. squash vs fixup

squash combines commits and lets you edit the resulting message. fixup combines commits while automatically using the earlier commit message.

```
git rebase -i HEAD~2
```

16. Git Tags

62. What is a Git tag?

A tag identifies a specific commit, commonly for releases.

```
git tag v1.0.0
git tag
git show v1.0.0
```

63. What is Semantic Versioning?

Semantic Versioning follows MAJOR.MINOR.PATCH. MAJOR represents breaking changes, MINOR backward-compatible features, and PATCH backward-compatible fixes.

64. Lightweight vs annotated tag

Lightweight tags are simple references. Annotated tags contain metadata such as tagger, date, and message and are generally preferred for releases.

```
git tag v1.0.0
git tag -a v1.0.0 -m "Release version 1.0.0"
git show v1.0.0
```

65. How do you push tags?

Push one tag or all tags to the remote.

```
git push origin v1.0.0
git push origin --tags
git push origin --delete v1.0.0
```

17. Git Internals

66. How does Git store data?

Git uses content-addressable objects. Important object types are Blob, Tree, Commit, and Tag.

67. What is a Git blob?

A blob stores the contents of a file. It does not directly store the filename.

68. What is a Git tree?

A tree represents directory structure and maps filenames to blobs and directories to trees.

69. What is a Git commit object?

A commit contains metadata and points to a snapshot tree and parent commit(s), along with author, committer, and message information.

70. Why does Git use hashing?

Hashing provides content addressing, object identification, integrity checking, and efficient lookup.

71. What does git hash-object do?

It calculates a Git object hash. With -w it can write the object into Git's object database.

```
git hash-object file.txt
git hash-object -w file.txt
```

72. What does git cat-file do?

It lets you inspect Git objects and determine their type.

```
git cat-file -t <hash>
git cat-file -p <hash>
git cat-file -p HEAD
```

18. Git Reflog

73. What is Git reflog?

Reflog records local movements of references such as HEAD and branch pointers. It is useful for recovery after resets, rebases, and accidental branch movement.

```
git reflog
```

74. Accidentally ran git reset --hard. Can you recover?

Often yes, if the previous commit is still referenced by the reflog. Create a recovery branch at the desired commit.

```
git reflog
git switch -c recovery abc123
```

75. Is reflog available on GitHub?

Reflog is primarily a local Git mechanism and is not a GitHub backup facility.

76. Can reflog recover everything?

No. Reflog entries can expire and unreachable objects can eventually be garbage-collected. It is a recovery mechanism, not a permanent backup.

19. Advanced Git Scenarios

77. You committed to the wrong branch. What would you do?

Find the commit, switch to the correct branch, and cherry-pick it. Then clean up the wrong branch if appropriate.

```
git log --oneline
git switch correct-branch
git cherry-pick <commit>
```

78. Uncommitted changes but need to switch branches?

Use stash or make a temporary commit if the changes are logically complete.

```
git stash
git switch other-branch
git stash pop
git add .
git commit -m "WIP"
```

79. Accidentally pushed a bad commit to main?

For a shared branch, generally revert the bad commit rather than rewriting shared history.

```
git revert <commit>
git push origin main
```

80. When would you use force push?

Usually after rewriting history on a private feature branch, such as after interactive rebase. Prefer --force-with-lease.

```
git push --force-with-lease
git push --force
```

81. What is git push --force-with-lease?

It force-updates the remote branch only when the remote branch is still at the expected state, reducing the risk of overwriting unseen work.

```
git push --force-with-lease
```

20. Senior-Level Practical Questions

82. Feature branch is behind main. How do you update it?

Use merge or rebase. For a private feature branch, rebase is often preferred for a cleaner history.

```
git fetch origin
git switch feature/login
git merge origin/main
git rebase origin/main
```

83. Multiple developers work on the same feature. Would you rebase?

Be careful with shared branches. Avoid casually rewriting history that other developers depend on; use merge or a team-approved strategy.

84. Developer says 'I always use git pull.' Concern?

Pull fetches and integrates immediately. For important branches, fetch first so you can inspect incoming changes before deciding whether to merge or rebase.

```
git fetch origin
git log HEAD..origin/main
git diff HEAD origin/main
git merge origin/main
git rebase origin/main
```

85. How would you maintain clean Git history?

Use feature branches, atomic commits, meaningful messages, Pull Requests, code reviews, CI checks, protected main, squashing where appropriate, rebase private branches, and avoid force pushes on shared branches.

86. Git workflow for a React/Next.js team

Use a protected main branch, feature branches, atomic commits, Pull Requests, code review, CI, and controlled merges.

```
git switch main
git pull
git switch -c feature/login
git status
git add .
git diff --staged
git commit -m "Add login form validation"
git push -u origin feature/login
```

21. Rapid-Fire Interview Questions

87. git add vs git commit

git add stages changes. git commit records the staged snapshot.

```
git add .
git commit -m "Add feature"
```

88. git fetch vs git pull

fetch downloads changes. pull fetches and then integrates them.

```
git fetch origin
git pull
```

89. git merge vs git rebase

Merge preserves history and may create a merge commit. Rebase rewrites commits onto a new base.

```
git merge main
git rebase main
```

90. git reset vs git revert

Reset moves references and can rewrite history. Revert creates a new commit that undoes another commit.

```
git reset HEAD~1
git revert HEAD
```

91. git restore vs git reset

Restore primarily handles file/staging restoration. Reset primarily moves HEAD/branch references and can also affect staging and working files.

```
git restore file.js
git reset HEAD~1
```

92. git stash pop vs git stash apply

pop applies and removes the stash. apply applies but keeps it.

```
git stash pop
git stash apply
```

93. What is HEAD?

HEAD represents your current checked-out position.

```
git status
git log -1
```

94. What is origin?

origin is the conventional name for the default remote repository.

```
git remote -v
```

95. What is origin/main?

A remote-tracking reference representing the last fetched state of the remote main.

```
git branch -r
```

96. What is a detached HEAD?

HEAD points directly to a commit rather than a branch.

```
git checkout <commit>
git switch -c recovery
```

97. What is a merge conflict?

A situation where Git cannot automatically reconcile competing changes.

```
git status
git add .
git commit
git merge --abort
```

98. What is cherry-pick?

It applies the changes introduced by a specific commit onto the current branch.

```
git cherry-pick <commit>
git cherry-pick --abort
git add .
git cherry-pick --continue
```

99. What is reflog?

A local record of reference movements that can help recover lost commits.

```
git reflog
```

100. What is interactive rebase?

A tool for modifying a sequence of commits, including squashing, rewording, fixing up, dropping, and reordering.

```
git rebase -i HEAD~5
```

22. Must-Know Git Commands

Configuration

```
git config --global user.name "Name"
git config --global user.email "email"
```

Repository

```
git init
git clone <url>
git status
```

Changes

```
git add .
git add <file>
git diff
git diff --staged
```

Commit

```
git commit -m "message"
git commit --amend
```

History

```
git log
git log --oneline --graph --decorate --all
git show <commit>
```

Branch

```
git branch
git branch -a
git switch -c feature/login
git switch main
git branch -d feature/login
```

Merge

```
git merge feature/login
git merge --abort
```

Rebase

```
git rebase main
git rebase -i HEAD~5
git rebase --continue
git rebase --abort
```

Stash

```
git stash
git stash list
git stash pop
git stash apply
```

Undo

```
git restore file.js
git restore --staged file.js
git reset --soft HEAD~1
git reset --mixed HEAD~1
git reset --hard HEAD~1
git revert <commit>
```

Remote

```
git remote -v
git remote add origin <url>
git fetch origin
git pull
git push origin main
git push -u origin feature/login
```

Remote branch

```
git branch -r
git push origin --delete feature/login
```

Cherry-pick

```
git cherry-pick <commit>
```

Tags

```
git tag v1.0.0
git tag -a v1.0.0 -m "Release v1.0.0"
git push origin v1.0.0
git push origin --tags
```

Recovery

```
git reflog
git switch -c recovery <commit>
```

Advanced

```
git blame <file>
git bisect start
git bisect bad
git bisect good <commit>
git clean -n
git clean -fd
```

23. Extra Senior-Level Commands

git blame

Shows who last changed each line of a file. Useful when investigating when and where a line was introduced.

```
git blame src/components/Login.tsx
```

git bisect

A binary-search tool for finding the commit that introduced a bug.

```
git bisect start
git bisect bad
git bisect good <known-good-commit>
git bisect good
git bisect bad
git bisect reset
```

git clean

Removes untracked files/directories. Preview first because deletion can be permanent.

```
git clean -n
git clean -f
git clean -fd
```

git show

Displays details of a commit, tag, or other Git object.

```
git show <commit>
git show HEAD
git show v1.0.0
```

24. Senior Git Scenarios to Practice

Priority scenarios

Practice these as spoken answers: accidental .env commit; bad commit pushed to main; branch behind main; uncommitted changes before switching; wrong-branch commit; accidental reset --hard; lost commit recovery; merge conflict; shared-branch rebase; force-with-lease; clean Git history; HEAD vs origin/main vs local main; merge vs rebase; git bisect; complete feature-branch → PR → CI → merge workflow.